

TreeLock: Merkle Tree-Based Inheritance in Decentralized Module Systems

S. C. Dunn
Starlabs, LLC
Colorado Springs, CO, USA
starlabsactual@gmail.com

Abstract

Modern software module systems face challenges in securely and efficiently resolving dependencies across decentralized and evolving codebases. This paper introduces a novel Merkle tree-based inheritance and resolution model that enables secure, deterministic, and incremental module finalization by capturing complete ancestry traces in a cryptographically verifiable manner. My approach formalizes symbol resolution and override semantics over inheritance DAGs, allowing for conflict detection and transparent override traceability. I present a scalable resolution algorithm and demonstrate its theoretical scalability through synthetic benchmarks simulating diverse module graph topologies. To validate feasibility, I outline a prototype implementation leveraging decentralized lookup protocols and WebAssembly-based evaluation. This model supports improved cacheability, fault tolerance, and tooling integration, positioning it as a promising foundation for future secure and decentralized module management systems.

CCS Concepts: • **Software and its engineering** → **Software configuration management and version control systems**; *Compilers*; *Distributed programming languages*; • **Theory of computation** → *Program semantics*; *Type structures*; • **Security and privacy** → *Cryptographic protocols*.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

Conference'17, Washington, DC, USA

© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM
<https://doi.org/10.1145/nnnnnnn.nnnnnnn>

Keywords: content-addressed modules, Merkle trees, decentralized registries, module resolution, module inheritance, type safety, generics, structural typing, inheritance, semantic versioning, reproducibility, distributed systems

ACM Reference Format:

S. C. Dunn. 2025. TreeLock: Merkle Tree-Based Inheritance in Decentralized Module Systems. In . ACM, New York, NY, USA, 23 pages. <https://doi.org/10.1145/nnnnnnn.nnnnnnn>

1 Introduction

Modern software development depends on large collections of independently authored modules. These modules evolve separately, follow distinct versioning patterns, and often depend on one another through complex relationships. Despite this complexity, developers expect module systems to provide predictable behavior, safe composition, and reproducible builds.

In this paper, I introduce a new approach to module resolution that treats each module as a symbolic commitment to its entire ancestry and override history. Instead of resolving names dynamically or relying on external registries to determine behavior, the system finalizes modules into cryptographic Merkle roots that represent their complete inheritance trace. These roots capture not only the content of the module but also the structure of its dependencies, the override relationships between symbols, and the path taken to produce each result.

This approach reframes module resolution as a form of symbolic hashing. The system walks an acyclic inheritance graph, tracks override sites, and computes a canonical resolution trace. The result is both deterministic and portable. Finalized modules can be distributed, cached, or compared without requiring access to their source trees. Symbol lookups are fast and reproducible, and resolution failures can be traced and understood in symbolic form.

The prototype implementation, called Tangl, demonstrates these ideas in practice [8]. It supports decentralized resolution using content-addressed inheritance and produces deterministic override traces. Although early in development, the tool is capable of resolving complex module graphs and is designed to integrate with existing

development workflows such as continuous integration and version control.

The main contributions of this work are:

- A symbolic module model based on immutable inheritance graphs
- A deterministic resolution algorithm that produces Merkle roots and override traces
- A reference implementation that supports decentralized caching and symbolic analysis
- An evaluation using synthetic module topologies to demonstrate scalability and reuse

This model enables a new kind of module system. It is one where inheritance is explicit, resolution is verifiable, and every module carries a cryptographic trace of its evolution. Detailed pseudocode for finalization and full mechanized proofs are provided in the supplementary materials.

2 Background

Decentralized module systems must support independently authored components that interoperate without a central authority. Traditional module systems rely on mutable registries or version constraints to locate and integrate dependencies. This leads to several challenges:

- **Ambiguity in resolution.** Different dependency paths or version constraints can yield varying outcomes, undermining reproducibility.
- **Trust and integrity.** Without content hashes, clients cannot independently verify that retrieved modules match expected code.
- **Conflict management.** Multiple inheritance paths may define the same symbol, requiring ad hoc override rules.
- **Scalability.** Large dependency graphs strain centralized services and complicate caching.

Merkle trees provide a simple remedy [2, 11]. By organizing module identities and their ancestry in a binary hash tree, clients obtain:

- *Content-addressed integrity*, since any modification alters a simple root hash.
- *Deterministic inclusion proofs*, enabling efficient verification of ancestry relationships.
- *Immutable references*, which improve cacheability and support decentralized resolution.

In the next section, I compare existing content-addressed storage and versioning approaches and highlight the gap that my Merkle tree-based inheritance model fills.

3 Related Work

Several systems employ content addressing to ensure integrity and reproducibility. Git uses Merkle DAGs of

commits and trees to track source changes with deduplication and history verification [4]. IPFS extends this idea to arbitrary file content, enabling decentralized storage and retrieval by hash [2]. Package managers such as npm, Cargo, and pip record version constraints and use lockfiles to approximate reproducible builds, but they still rely on mutable registries and name-based resolution, which can lead to dependency ambiguity and “dependency hell” [5, 13, 16].

Git and IPFS focus on content provenance and distribution without first-class support for symbol resolution or compositional semantics. npm, Cargo, and pip address version conflicts via constraint solvers and lockfiles rather than explicit override traces, making deterministic resolution challenging in complex, multi-parent inheritance scenarios. Existing dependency systems therefore lack a unified, cryptographically verifiable mechanism for module inheritance and override semantics within a content-addressed framework.

3.1 Motivation and Practical Benefits

Beyond formal guarantees of determinism and integrity, Merkle tree-based inheritance delivers concrete advantages in real-world software development:

- **Reproducible builds:** By content-addressing every module and its full transitive ancestry, builds become byte-for-byte identical across machines and over time, eliminating the “it works on my machine” problem.
- **Fine-grained patching:** Teams can apply small local fixes or security patches by inheriting and overriding only the affected symbols, rather than forking entire repositories, while preserving full provenance and auditability.
- **Supply-chain security:** Cryptographic hashing of module graphs makes tampering immediately detectable. Downstream consumers can verify that every dependency, direct or transitive, matches the published Merkle roots before execution.
- **Offline and CDN-friendly workflows:** Since every module is identified solely by its hash, clients can cache and serve artifacts via CDNs or peer-to-peer networks, with no reliance on a central registry for version lookups.
- **Incremental CI/CD:** Continuous integration systems can skip re-finalizing or re-testing subtrees of the graph that haven’t changed (their Merkle roots remain the same), dramatically reducing build times.
- **Interoperability with existing ecosystems:** Semver-based imports are translated to Merkle roots at finalization, allowing gradual adoption without rewriting existing package manifests.

These practical benefits—combined with the formal metatheory—make the Merkle tree-based inheritance model not only provably sound but also immediately valuable for modern, decentralized software supply chains.

4 Challenges Addressed by Merkle Tree-Based Inheritance

Traditional inheritance and dependency models face several core challenges:

- **Diamond problem.** Multiple inheritance paths can yield duplicated or conflicting definitions [14].
- **Resolution ambiguity.** Name conflicts often require manual disambiguation or complex solver logic.
- **Cyclic dependencies.** Unchecked cycles lead to nonterminating resolution and runtime errors [2].
- **Tamper risk.** Mutable identifiers (names or versions) allow code substitution attacks or accidental drift.
- **Inefficient validation.** Centralized lookups and repeated ancestry computation hinder caching and scalability.

Merkle tree-based inheritance overcomes these issues by:

- Representing all transitive ancestors in a single DAG; shared ancestors collapse naturally, eliminating duplicates.
- Enforcing a canonical, hash-based ordering of ancestors to resolve conflicts deterministically.
- Preventing cycles via acyclic hash dependencies; any cycle would invalidate the Merkle proof.
- Binding module identity to content and ancestry roots, making any change immediately detectable.
- Allowing cached subtrees of the DAG and amortized ancestry hashing for scalable resolution.

5 Comparison with Existing Systems

Traditional package managers and version control tools address parts of the module management problem:

- **npm, Cargo, pip.** Use name-and-version constraints with lockfiles. They resolve conflicts via solvers and lockfiles but lack explicit override traces, making multi-parent inheritance brittle [5, 13, 16].
- **Git.** Tracks content history in a Merkle DAG but focuses on commits and files rather than symbol-level composition [4].
- **IPFS.** Provides decentralized content addressing and retrieval but does not formalize module composition or inheritance semantics [2].

- **Nix.** Achieves reproducible builds with content-addressed derivations but targets whole-system environments, not modular symbol resolution [6].

By contrast, my Merkle tree-based inheritance model:

- Treats modules as cryptographic commitments to both code and resolved ancestry.
- Provides first-class support for symbol overrides and multi-parent inheritance in a content-addressed setting.
- Ensures deterministic resolution without external solvers or mutable metadata.
- Enables scalable caching of shared subtrees and on-demand validation of ancestry proofs.

6 System Design

6.1 Finalization

Finalization freezes a module M into the global inheritance DAG by resolving imports, incorporating parent ancestry (via parent hashes), and assigning a content-addressed identity. I define a total function

$$\mathcal{F}(C_M, T_M, A_M, I_M) = h_M$$

that proceeds as follows:

1. **Resolve imports.** For each symbolic reference $r \in I_M$, look up the finalized parent ID $h_P = \mathcal{R}(r)$.
2. **Collect direct parents.** Let

$$P = \{h_{P_1}, \dots, h_{P_k}\}.$$

Each h_{P_i} already encodes its own transitive ancestry.

3. **Compute parent Merkle root.** Sort P lexicographically and build a balanced Merkle tree; let

$$m = H_{\text{merkle}}(P).$$

Figure 1 illustrates how those parent roots become leaves in the new tree.

4. **Content hash.** Compute

$$h_C = H(C_M).$$

5. **Final module ID.** Return

$$h_M = H(h_C \parallel T_M \parallel A_M \parallel m).$$

Because each parent hash encapsulates its full ancestry, step 2 implicitly covers all transitive ancestors without explicit recursion. This guarantees that any change to M or its ancestors yields a different h_M , ensuring determinism and tamper-proof integrity.

6.2 Recursive Module Groups

To support mutually recursive modules without breaking our DAG-based Merkle finalization, we collapse each strongly-connected component (SCC) of the import graph into an *atomic group* G :

1. **SCC detection.** Perform a static analysis on the symbolic import graph to identify each SCC of modules that (directly or indirectly) import one another.

2. **Group serialization.** For each SCC

$$G = \{M_1, \dots, M_n\}$$

, collect:

- Each module's content C_{M_i} .
- The intra-group import edges.

Serialize these in a canonical, order-insensitive form (e.g. sort by module name).

3. **Atomic hash.** Compute a single hash

$$h_G = H(\text{serialize}(G)).$$

Exports from G are now frozen and cannot be overridden by any module outside G .

4. **DAG collapse.** Replace all edges into any member of G with edges into the single node h_G . Upstream finalization and Merkle-tree inheritance then proceed over this acyclic, hash-identified DAG of groups and standalone modules.

At finalization time, only the hashes of direct parents, each of which may be an atomic group, are rehashed to form the new Merkle root. Since exports within G are frozen, no external overrides can alter its semantics or hash after h_G is computed.

As illustrated in Figure 2, atomic recursive groups are finalized as a single unit, ensuring consistent hashing and resolution, in contrast to non-atomic modules which are finalized independently.

Termination and Type Safety. We apply a light-weight *static termination check* to each G : a structural-recursion analysis flags any self-calls that do not operate on a strictly smaller argument. For more complex patterns, developers may supply external measure files (side-car metadata) that attest to well-founded recursion. Any failure in these checks prevents finalization of G , guaranteeing all recursive definitions within G terminate.

7 Use Cases and Interoperability

This section illustrates how Merkle tree-based inheritance enables both fine-grained overrides and seamless integration with existing ecosystems.

7.1 Overriding and Forking a Dependency

Suppose a developer depends on a module `libA` that exports a function `validate()`. Traditionally, patching `validate()` requires forking the entire package and updating downstream dependencies. With Merkle inheritance, the developer creates a new module `libB`:

Listing 1. Override of `validate()` in a new module

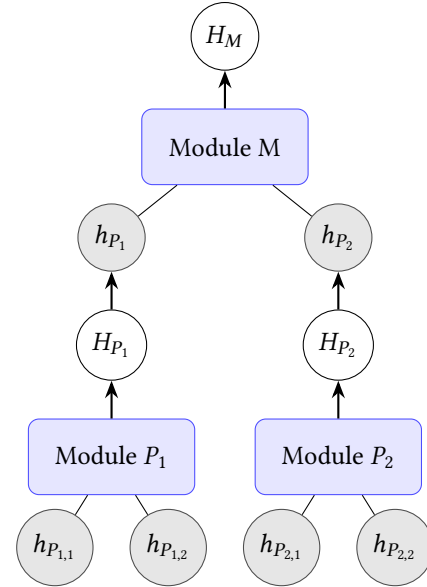


Figure 1. Recursive Merkle-tree inheritance: each module's finalized hash H_{P_i} (shown as an internal node) is itself a Merkle root over its own parents, and these hashes become the leaves of its child's tree.

Atomic Recursive Group Non-Atomic Modules

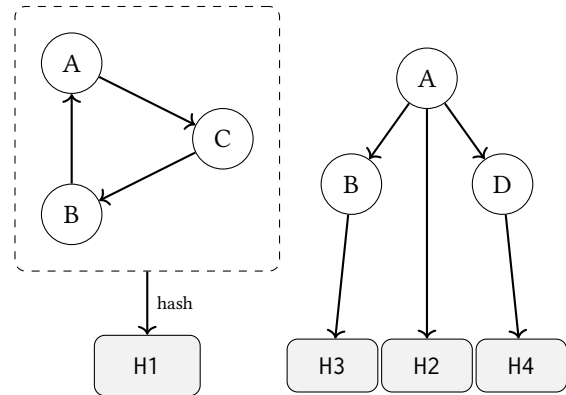


Figure 2. Atomic recursive modules are grouped and hashed as a single unit (H1), while non-atomic modules are hashed individually (H2, H3, H4).

```

1 {
2   % libA's finalized hash
3   "imports": ["merkle:abcd...1234"],
4   "exports": {
5     "validate": /* new implementation */
6   }
7 }

```

During finalization, `libB` reuses all symbols from `libA` except `validate`. Clients can verify this override by

inspecting the Merkle ancestry proof. Figure 6 illustrates conflict resolution rules in a similar context.

7.2 Semver Translation Workflow

Many projects declare dependencies with semver ranges (e.g. “^1.3.0” in npm or Cargo). To interoperate, an import adapter resolves these ranges against the native ecosystem registry, then replaces them with concrete Merkle roots:

1. Read package.json or Cargo.toml.
2. Use npm/Cargo to select version 1.3.5.
3. Query the Merkle registry:

$$(\text{libA}, 1.3.5) \xrightarrow{\mathcal{R}} \text{merkle:abcd} \dots 1234.$$

4. Rewrite imports to merkle:abcd...1234 before finalization.

This workflow yields lockfile-free, content-addressed builds that remain reproducible.

7.3 Runtime Symbol Resolution

At execution time, symbols are resolved lazily via a Resolver class (Listing 3). The runtime consults a static reference map and fetches modules on demand. Figure 5 diagrams this process.

7.4 Registry Integration

A decentralized registry maps human-readable names and version constraints to Merkle roots:

$$\mathcal{R} : (\text{namespace}, \text{name}, \text{semver}) \mapsto h_M.$$

Clients verify digital signatures on namespace Merkle roots to ensure only authorized publishers can update entries. This design supports:

- *Exact-version lookup*, ensuring reproducibility.
- *Alternative registries*, allowing private or mirrored namespaces.
- *Auditability*, via signed Merkle proofs of registry bindings.

8 Merkle Tree-Based Inheritance Model

I now present the core inheritance model built on Merkle trees and content addressing. Each module’s direct parent hashes already embed their full ancestry, so finalization need only rehash those parent IDs.

8.1 Merkle Trees

A Merkle tree is a binary hash tree in which each leaf is the hash of a data element and each internal node is the hash of its two children. This enables:

- *Efficient inclusion proofs*, via logarithmic-sized authentication paths.
- *Tamper detection*, since any change to a leaf or subtree propagates to the root.

8.2 Content-Addressed Modules

Each module M consists of content C_M and a (possibly empty) set of symbolic imports I_M . Modules are identified by:

$$h_C = H(C_M), \quad h_M = H(h_C \parallel H_{\text{merkle}}(P)),$$

where P is the set of finalized parent hashes. Because each parent hash h_{P_i} already encodes its own ancestry, hashing P suffices to capture the entire transitive closure. As shown in Figure 1, each parent hash H_{P_i} is itself computed by hashing its own parents, and then is used as a leaf in module M ’s tree.

8.3 Inheritance Mechanism

To inherit from parent modules:

1. Resolve each import $i \in I_M$ via the registry
 $\mathcal{R}(i) \mapsto h_{P_i}$.
2. Collect direct parents $P = \{h_{P_1}, \dots, h_{P_k}\}$.
3. Sort P lexicographically and build a balanced Merkle tree to compute

$$m = H_{\text{merkle}}(P).$$

4. Compute the final ID

$$h_M = H(h_C \parallel m).$$

8.4 Canonicalization of Ancestry

Since each h_{P_i} is itself a Merkle root over its parents, this process efficiently and deterministically captures all transitive ancestors without explicit recursion. The final module ID h_M is a hash of both the content and the Merkle tree of its parents, ensuring that any change to M or its ancestors yields a different h_M . This guarantees reproducibility and integrity. Figure 1 illustrates the recursive nature of this inheritance model.

8.5 Symbol Resolution Semantics

I view symbol resolution as a total function over the Merkle DAG: shadowing and override precedence follow a deterministic, breadth-first merge of ancestors. Each lookup records a symbolic trace of visited modules, making conflict resolution transparent and analyzable [1, 7]. Formal judgments and proofs of soundness and determinism appear in the next section.

8.6 Atomic Recursive Modules and Resolution Edge Cases

The module system is built on Merkle tree-based inheritance, where each module is identified by a cryptographic hash of its resolved contents. This ensures reproducibility, immutability, and safe composition. To support recursive structures within this framework, we introduce the concept of *atomic recursive modules*, which

allow internally self-referential definitions without violating the acyclic nature of the Merkle graph.

8.6.1 Recursive Modules via Atomic Grouping. In standard Merkle tree semantics, recursive definitions pose a challenge because a module cannot refer to itself directly as doing so would prevent computing its hash. To address this, recursive modules are treated as *atomic groups*: instead of resolving definitions incrementally, the entire group is resolved as a single unit and hashed only after all internal references are finalized.

This allows the system to support recursive structures like trees, lists, and graphs. For example, a binary tree can be defined recursively using a self-referential module that is resolved atomically (see subsection 8.8). The atomic group ensures that all internal references are scoped and validated together, preventing any external circular dependencies.

While modules within an atomic group may reference each other recursively, the group as a whole is finalized before it can be linked elsewhere. This preserves the global acyclicity of the Merkle dependency graph. Any cycle involving multiple atomic groups is rejected at resolution time.

8.6.2 Conflicting Module Versions. Conflicting versions of a dependency often lead to ambiguity or runtime bugs in traditional systems. Here, version conflicts are handled cleanly through content-addressing. Because each module version yields a different hash, they are treated as distinct modules by the system.

Multiple versions of a module can be imported and used independently:

```
import "crypto@1.2.0" as crypto_v1;
import "crypto@2.0.1" as crypto_v2;
```

These modules coexist safely: their hashes, scope, and contents are isolated, and the system performs no implicit merging. When compiled to sandboxed targets like WebAssembly, their runtime separation is further enforced at the memory and execution boundary.

Tools can analyze the Merkle graph to report redundant or shadowed versions, but this is an opt-in ergonomic aid. The underlying resolution remains deterministic and transparent: different hashes mean different modules, regardless of name similarity or structural overlap.

Summary. Atomic recursive modules make recursive definitions first-class citizens within a Merkle tree-based module system. They preserve the resolution guarantees of immutability and acyclic structure, while enabling the expressive power needed for self-referential data types and algorithms. At the same time, versioning remains unambiguous and composable, supporting robust modular reuse in large and complex systems.

8.7 Worked Example: Symbol Resolution and Override Tracing

To demonstrate how finalization and symbol resolution interact with override tracing, consider the following module hierarchy:

Listing 2. Module definitions for resolution example

```
module A {
  val x = 1
  val y = 2
}

module B inherits A {
  val y = 3
  val w = x + y
}

module C inherits A {
  val z = y + 10
}

module D inherits B, C {
  val u = z + w
}
```

Finalization proceeds bottom-up:

- Module A has no parents and is finalized first.
- Module B inherits from A, overrides y, and defines w.
- Module C also inherits from A, reuses y, and defines z.
- Module D inherits from both B and C, and defines u.

When resolving the symbol u in D, the override trace proceeds as follows:

- u depends on z and w.
- z is defined in C, which inherits y from A, so z resolves to $2 + 10 = 12$.
- w is defined in B and depends on x and the overridden y, so w resolves to $1 + 3 = 4$.
- u therefore resolves to $12 + 4 = 16$.

The trace for u shows multiple inheritance paths and captures symbol provenance:

$$u \mapsto z + w \Rightarrow \begin{cases} z \in C \Rightarrow y \in A \\ w \in B \Rightarrow x \in A, y \text{ overridden in B} \end{cases}$$

This trace confirms:

- Deterministic symbol resolution even in the presence of overrides.
- Accurate ancestry mapping via finalized hashes.

- That override points are explicitly captured in the trace.

The finalized hash of module D includes the finalized hashes of both B and C, recursively covering the full ancestry tree. This enables cryptographic validation of symbol provenance and ensures deterministic rebuilds.

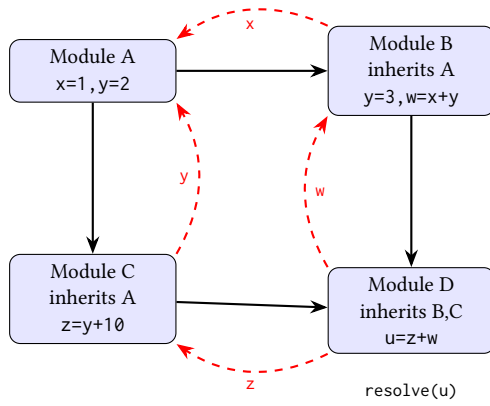


Figure 3. Symbol resolution for u in Module D. Solid arrows show inheritance; dashed red arrows trace how $u = z + w$ resolves through z in C (then y in A) and w in B (then x in A).

Figure 3 illustrates how the symbol u in Module D is resolved:

1. D inherits from B and C.
2. The dashed red arrow from D to C shows that z is looked up in C, which in turn inherits y from A.
3. The dashed red arrow from D to B shows that w is looked up in B, which uses the overridden y from B and x from A.
4. Ultimately, $u = z + w$ resolves to values computed from A's and B's definitions, and the Merkle finalization of D incorporates the finalized hashes of B and C (which themselves embed A), ensuring full transitive integrity.

8.8 Worked Example: A Recursive Module for Binary Trees

Recursive modules are essential for defining tree-like structures, where each node can contain subtrees of the same type. In this example, we use a recursive module to define a binary tree of integers.

Module Definition. We define a module named `tree` that contains a class `Tree`. This class represents either a `Leaf`, or a `Node` containing an integer value and two child trees.

```
# Module: tree
class Leaf:
    pass
```

```
class Node:
```

```
def __init__(
    self,
    value: int,
    left: 'Tree',
    right: 'Tree'
):
    self.value = value
    self.left = left
    self.right = right
```

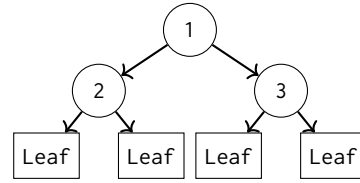
`Tree = Leaf | Node`

This structure is recursive: a `Node` refers to instances of `Tree`, which can themselves be `Nodes` or `Leafs`.

Example Construction. Here is how we can construct a simple binary tree with three nodes:

```
t = Node(1,
    Node(2, Leaf(), Leaf()),
    Node(3, Leaf(), Leaf()))
```

Visual Representation. The structure of this tree can be visualized as follows:



Recursive Rule. With the recursive structure in place, we can define a recursive function to count the number of nodes in the tree:

```
def size(t: Tree) -> int:
    if isinstance(t, Leaf):
        return 0
    elif isinstance(t, Node):
        return 1 + size(t.left) + size(t.right)
```

This function uses standard Python-style pattern matching with `isinstance`, and recursively counts all the internal `Nodes` in the tree.

Summary. This example illustrates how recursive data structures can be implemented cleanly using modules. By using classes to define variants like `Leaf` and `Node`, and recursive references to the type `Tree`, we can express complex hierarchical data naturally.

9 Formal Module Semantics

This section states the key judgments, inference rules, and metatheoretic properties of symbol resolution and override traces.

9.1 Resolution Judgments and Inference Rules

$$M \vdash s \Downarrow d \quad \text{and} \quad M \vdash s \Downarrow d \rightsquigarrow \pi$$

where π is the override trace. The rules are:

$$\begin{array}{c}
\frac{s \in \text{dom}(\Gamma_M) \quad \Gamma_M(s) = d}{M \vdash s \Downarrow d} \quad (\text{LOCAL}) \\
\\
\frac{P_i \vdash s \Downarrow d \quad \forall j < i. s \notin \text{dom}(\Gamma_{P_j})}{M \vdash s \Downarrow d} \quad (\text{INHERIT}) \\
\\
\frac{s \in \text{dom}(\Gamma_M)}{M \vdash s \Downarrow d \rightsquigarrow [M]} \quad (\text{T-LOC}) \\
\\
\frac{P_i \vdash s \Downarrow d \rightsquigarrow \pi \quad \forall j < i. s \notin \text{dom}(\Gamma_{P_j})}{M \vdash s \Downarrow d \rightsquigarrow [M] ++ \pi} \quad (\text{T-INH})
\end{array}$$

9.2 Metatheoretic Properties

Determinism. *Theorem.* If $M \vdash s \Downarrow d_1$ and $M \vdash s \Downarrow d_2$, then $d_1 = d_2$.

Typing Judgment (Sketch). Each module M has a typing context Γ_M . I write

$$\Gamma \vdash M : \text{OK}$$

to mean all definitions and references in M are type-consistent. Child overrides may freely redefine types; consumers of M perform verification upon import.

Soundness, Compositionality. Resolution is sound (every resolved symbol is a valid definition), compositional (combining DAG fragments preserves resolution outcomes), and deterministic (no nondeterminism in lookup). Detailed proofs are in the appendix.

9.3 Worked Formal Example

We illustrate the formal resolution judgment

$$M \vdash s \Downarrow d \rightsquigarrow \pi$$

using the modules from our worked example:

- A exports $\{x, y\}$.
- B inherits A and overrides y .
- C inherits A without overrides.
- D inherits B, C .

We show the derivation of

$$D \vdash x \Downarrow (A, x) \rightsquigarrow [D, B, A]$$

using rules T-LOC and T-INH:

$$\begin{array}{c}
\text{T-LOC-A} \\
\frac{x \in \text{dom}(\Gamma_A) \quad \Gamma_A(x) = (A, x)}{A \vdash x \Downarrow (A, x) \rightsquigarrow [A]} \\
\\
\text{T-INH-B} \\
\frac{A \vdash x \Downarrow (A, x) \rightsquigarrow [A] \quad \forall j < 1. x \notin \text{dom}(\Gamma_{P_j})}{B \vdash x \Downarrow (A, x) \rightsquigarrow [B] ++ [A]}
\end{array}$$

$$\begin{array}{c}
\text{T-INH-D} \\
\frac{B \vdash x \Downarrow (A, x) \rightsquigarrow [B, A] \quad \forall j < 1. x \notin \text{dom}(\Gamma_C)}{D \vdash x \Downarrow (A, x) \rightsquigarrow [D] ++ [B, A]}
\end{array}$$

Here:

- Rule T-LOC-A applies in A since x is locally defined.
- Rule T-INH-B lifts the result into B , skipping over any earlier parents that do not define x .
- Rule T-INH-D finally lifts into D , yielding the full override trace $\pi = [D, B, A]$.

This derivation demonstrates:

1. *Determinism*: each step picks the first ancestor defining x .
2. *Trace capture*: the list $[D, B, A]$ records the chain of modules visited.
3. *Compositionality*: resolution in D reuses the derivation in B .

10 Lean Theorems

I state the core properties of the Merkle hashing and override-trace mechanisms. Full mechanized proofs appear in Appendix E.

Theorem 10.1 (Deterministic Merkle Hashing). *For any sorted list of hashes L , all balanced Merkle-tree constructions over L yield the same root:*

$$\forall T_1, T_2. \text{treeHash}_{T_1}(L) = \text{treeHash}_{T_2}(L).$$

Proof sketch. Both constructions only reorder and hash pairs of elements in L , and SHA-256 is collision-resistant. See Appendix A.1 for the full proof.

Lemma 10.2 (List-to-Tree Correctness). *For any sorted list L ,*

$$\text{leaves}(\text{listToTree}(L)) = L.$$

Proof sketch. By induction on the balanced-tree construction function, which preserves input order at the leaves. See Appendix A.2.

Theorem 10.3 (Compositional Merkle Hash). *For any two (possibly overlapping) sets of parent hashes P_1 and P_2 ,*

$$H_{\text{merkle}}(P_1 \cup P_2) = H(H_{\text{merkle}}(P_1) \parallel H_{\text{merkle}}(P_2)).$$

Proof sketch. Shared elements appear exactly once in the union and in both subtrees; merging two roots preserves the canonical structure. See Appendix A.3.

Proposition 10.4 (Override Trace Finiteness). *For every symbol s in a finalized module, its override trace $\pi = \text{trace}(s)$ is a finite, acyclic sequence of module IDs.*

Proof sketch. Cycles are impossible because each group or module is frozen before hashing, and the import graph is collapsed to a DAG of atomic nodes. See Appendix A.4.

11 Formalization of Finalization

Before computing Merkle roots, the system first collapses any mutually recursive modules into atomic groups. Formally:

1. **SCC collapse.** Given the symbolic import graph, compute its strongly-connected components (SCCs). Treat each SCC G as a single atomic node whose content is the canonical serialization of all modules in G .
2. **Resolve imports.** For each remaining symbolic reference r , look up its finalized parent hash $h_P = \mathcal{R}(r)$, where P includes both standalone modules and atomic groups.
3. **Collect direct parents.** Let

$$P = \{h_{P_1}, \dots, h_{P_k}\}.$$

Each h_{P_i} already encodes its own transitive ancestry (and any recursion is contained within its atomic group).

4. **Compute parent Merkle root.** Sort P lexicographically and build a balanced Merkle tree; let

$$m = H_{\text{merkle}}(P).$$

5. **Content hash.** Compute

$$h_C = H(C_M).$$

6. **Final module ID.** Return

$$h_M = H(h_C \parallel T_M \parallel A_M \parallel m).$$

This procedure remains fully static and deterministic. By collapsing recursive groups before hashing, we preserve an acyclic DAG for Merkle-tree finalization while still supporting mutual recursion internally.

Key Property. Since each direct parent hash h_{P_i} already encodes its full ancestry, step 3 suffices to guarantee that h_M reflects all transitive ancestors without needing an explicit closure computation.

11.1 Determinism and Injectivity

Determinism. Repeated application of $\mathcal{F}$ on the same inputs yields the same h_M , since all operations (registry lookup, sorting, hashing) are deterministic.

Injectivity. If $\mathcal{F}(X_1) = \mathcal{F}(X_2)$ then $X_1 = X_2$, by collision resistance of H and inclusion of all distinguishing components (C, T, A, P) in the final hash.

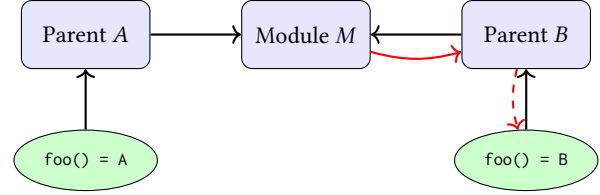


Figure 4. Symbol resolution in a Merkle-based inheritance graph. Module M inherits from parents A and B , with left-to-right override precedence. The arrows illustrate the path taken to resolve the symbol `foo`, which is found in both parents. Dotted red arrows represent resolution from the module to the symbol definition.

12 Type System Integration

While the system remains fundamentally language-agnostic, many modern ecosystems rely on type annotations and interface contracts to ensure safe composition. To support this, I propose a two-phase type integration model that preserves the flexibility of content-addressed inheritance while enabling static guarantees where needed.

Two-Phase Typing for Module Finalization. I introduce a two-phase typing model designed to catch type mismatches early while maintaining the independence and reusability of individual modules:

1. Pre-Finalization Type Annotation Phase:

Each module may declare its own exports and expected imports using optional type signatures. These annotations serve as structural contracts and can be checked in isolation without resolving the entire inheritance graph. This phase supports ecosystem-specific type formats (e.g., TypeScript, Flow, Rust types).

2. Post-Finalization Resolution Phase:

Once the full inheritance graph is resolved and finalized, the system performs type unification, verifying that re-exports and overrides preserve declared types. Any mismatches or violations of declared type bounds can be caught during this phase, ensuring that the final module is type-safe.

This approach preserves local reasoning for individual modules, enables partial type-checking during development, and supports eventual consistency and validation in the finalized view. It also avoids requiring global type inference, which may not be tractable across distributed registries.

Optional Typing During Module Resolution. Typed ecosystems can optionally perform type compatibility checks at resolution time. This includes verifying that

imported or overridden symbols maintain compatible types, checking for shadowing violations, and ensuring interface conformity for inherited modules.

Declarative Type Annotations. Modules may provide inline or sidcar type annotations, allowing downstream tools or runtimes to validate that symbols match their declared types. For example, if `moduleA` exports a function `compute()` with signature $\text{Int} \rightarrow \text{Int}$, any modules inheriting or re-exporting it must preserve this type unless explicitly widened or aliased.

Generic Type Support. Typed ecosystems may adopt *generic type aliases* and structural type constraints to define parametric modules or enforce interface-like requirements across inheritance trees. These mechanisms are sketched in Appendix C and align with the semantics of finalization and override resolution.

By enabling early detection of type mismatches and supporting structural compatibility guarantees, the system provides a pathway to scalable, decentralized type safety. This approach is particularly useful for ecosystems seeking to layer formal validation atop content-addressed distribution.

As discussed in Section 24, future work includes implementing richer type inference mechanisms, subtyping hierarchies, and generics-aware resolution, enabling even more robust type integration across distributed modules.

Connections to Module Calculi. My system is not itself a formal module calculus, but it draws on key ideas from foundational work on ML-style modules [12] and the mixin calculus [3]. In particular, it supports compositionality, deterministic symbol resolution, and (optionally) generic module interfaces. Detailed discussion of its relationship to these calculi, and the requirements for supporting features like higher-order modules and recursive types, is provided in the supplementary materials (Appendix D).

13 Namespace-Based Registry

To facilitate human-readable and interoperable module references, I introduce a decentralized registry system that maps symbolic names and version constraints to Merkle-based module identities. This registry layer provides a structured namespace for modules while preserving the immutability and transparency of the underlying content-addressed architecture.

13.1 Human-Readable Naming

To enable discoverability and developer ergonomics, modules may be referred to by symbolic identifiers of the form:

namespace/name@version

For example:

core/math@1.0.0

These identifiers are resolved to finalized module hashes via a distributed registry system.

13.2 Registry Semantics

I define a registry as a mapping:

$$\mathcal{R} : (\text{namespace}, \text{name}, \text{semver}) \rightarrow h_M$$

Each registry entry is a triple that maps a human-readable namespace and identifier and a semantic version constraint to a finalized module ID.

13.3 Version Constraints

The current system supports only *exact versioning* for modules. Modules must be referenced using fully specified semantic versions, such as `core/math@1.2.3`.

This ensures reproducibility and avoids ambiguity during resolution. Each version corresponds to a finalized module hash, and resolution is a direct lookup of that specific version in the distributed registry.

While this restriction simplifies caching, distribution, and verification, it limits the ability to express flexible or evolving dependencies. As future work, I plan to support more expressive version constraints, including:

- Compatible Versions: `@1.2.0`
- Version Ranges: `>=1.0.0 <2.0.0`

These forms would require additional mechanisms for compatibility analysis, deterministic resolution, and finalization tracking across evolving module sets.

13.4 Support for latest Tag in Module References

To improve developer ergonomics during early development and testing, the system supports symbolic module references using a reserved latest tag. This tag acts as a placeholder for the most recent version of a module available at the time of resolution, determined using a combination of metadata (e.g., version numbers, timestamps) or registry-specific heuristics.

Pre-Finalization Semantics. The latest tag is resolved during the pre-finalization phase of module compilation or bundling. At this stage, all symbolic references are dereferenced to content-addressed identifiers (e.g., hashes), ensuring that the finalized module graph remains immutable and reproducible. This allows developers to write:

```
import utils from "example.org/lib@latest";
which is resolved to:
import utils from "example.org/lib@1.2#hash";
based on the state of the distributed registry or a pinned
version map at the time of finalization.
```

Reproducibility and Pinning. To ensure builds remain deterministic, the resolution of `latest` must be pinned to a concrete content hash before module finalization. Once resolved, symbolic tags are no longer part of the module graph. Build systems and registries should retain a record of the mapping between symbolic tags and resolved identifiers to support reproducibility audits.

Limitations. Because the `latest` tag is inherently non-deterministic, its use is discouraged in finalized modules or published registries. It is intended for use in development workflows, rapid prototyping, and interactive environments such as REPLs or notebooks. Static analyzers and linters may issue warnings if `latest` appears in production code paths.

Related Features. The system may also support additional symbolic tags like `beta`, `stable`, or `semver-compatible` wildcards (e.g., `1.x`), resolved via similar mechanisms before finalization. All symbolic references must ultimately reduce to a content-addressed form to participate in the Merkle tree-based inheritance model.

13.5 Immutable Bindings

Once a registry entry is created, it is immutable. That is:

$$\mathcal{R}(n, m, v) = h_M \text{ is fixed for all time}$$

This ensures that symbolic references remain stable and verifiable over time.

13.6 Distributed Implementation

The registry is implemented as a distributed store backed by the same DHT infrastructure used for module exchange [15]. Each namespace is a signed collection of bindings, rooted by a Merkle tree [10]:

$$\text{namespace} \rightarrow H_{\text{merkle}}(\text{bindings})$$

Each binding is a tuple:

$$(\text{name}, \text{version}, h_M)$$

Namespaces may be published and updated only by owners who can sign new Merkle roots. Ownership is tracked via public key cryptography, and clients verify signatures before accepting updates.

13.7 Integration with Finalization

During module finalization, any symbolic parent reference of the form:

```
import core/math@1.0.0
```

is resolved through the registry:

$$(\text{core}, \text{math}, 1.0.0) \xrightarrow{\mathcal{R}} h_{M'}$$

The resulting module ID $h_{M'}$ is inserted into the parent set P , and all symbolic names are replaced by cryptographic hashes. This process is deterministic and ensures that the final module ID is fully derived from immutable data.

13.8 Registry Trust Model

Registries do not require global trust. Clients are free to:

- Pin specific hashes instead of using symbolic names.
- Use alternate or private registries for specific namespaces.
- Require signatures from known authors or authorities.

This flexibility allows the ecosystem to support both centralized and decentralized workflows while preserving verifiability.

14 Runtime and Resolution Semantics

In Tangl, modules are compiled into a platform-independent format (e.g., WebAssembly or JavaScript) and executed in a virtual machine (VM). A finalized module is fully deterministic: it includes a content-addressed identifier, a static resolution table, and a record of its transitive ancestry. At execution time, this structure enables on-demand, verifiable, and reproducible behavior.

14.1 Resolve Phase

Rather than preloading all dependencies, the Tangl VM performs a *resolve phase* prior to execution:

- It traverses the module's resolution map R_M , which maps symbol names to ancestor modules and specific exports.
- For each module ID referenced in R_M , it checks whether the corresponding module is available locally or locates it in the distributed network.
- A resolution index $\rho : h_{M_i} \mapsto \text{location}$ is constructed, mapping content hashes to their resolvable location (e.g., a local path, cached artifact, or network URI).

No modules are actually fetched or loaded during this phase. Instead, the resolve phase ensures that all necessary modules are *locatable*, enabling efficient runtime linking.

14.2 Lazy Loading at Runtime

Once resolution is complete, the module is executed with *lazy loading* semantics:

- Ancestor modules are not loaded upfront.
- When a symbol is first accessed during execution, the VM uses R_M to determine which ancestor module it belongs to.

- If the ancestor has not yet been loaded, the VM consults ρ to fetch, parse, and instantiate it on demand.
- Resolved exports are cached for subsequent use.

This strategy ensures minimal startup overhead, efficient caching, and deterministic behavior across environments.

14.3 Symbol Resolution Example

To illustrate the resolution logic, consider a module Main that inherits from two modules A and B, both of which export a symbol foo. Tangl's resolution map R_M determines which export is used based on two factors:

- **Import Order:** If both modules define the same symbol, the last one imported "wins" by default.
- **Import Granularity:** A symbol-specific import (e.g., `import(A.foo)`) takes precedence over whole-module imports.

Scenario 1: Whole-module imports, ordered B then A.

```
import(B);
import(A);
foo();
```

Even though both A and B export foo, the reference map R_{Main} resolves:

$$R_{Main}(foo) = \langle A, foo \rangle$$

because A was imported after B, overriding its binding.

Scenario 2: Symbol-specific import.

```
import(A.foo);
import(B);
foo();
```

Even though B was imported later, the reference map gives priority to the symbol-specific import:

$$R_{Main}(foo) = \langle A, foo \rangle$$

This allows developers to explicitly pin symbols to particular modules, regardless of the overall import order.

Scenario 3: No disambiguation provided.

```
import(A);
import(B);
foo();
```

This resolves to:

$$R_{Main}(foo) = \langle B, foo \rangle$$

since B was the last module imported that provided foo. Had both modules been imported in the reverse order, the binding would change.

Summary. Tangl's resolution rules provide both predictability and flexibility:

- The default rule is *last-imported wins*.
- Symbol-specific imports override full-module imports.
- Resolution is captured statically in R_M , which is persisted in the finalization step and guarantees reproducibility across machines and time.

This allows Tangl programs to explicitly disambiguate conflicting definitions and avoid ambiguous resolution paths without resorting to runtime heuristics.

Listing 3. Lazy symbol resolution in the Resolver class

```
class Resolver:
def __init__(
    self,
    reference_map,
    content_store,
    fetch_from_network
):
    # foo -> a29d...3c23
    self.reference_map = reference_map
    # Local content store
    self.cs = content_store
    # Fetch fn
    self.fetch = fetch_from_network
    # Memoized resolutions
    self.resolved_cache = {}

def resolve(self, symbol):
    if symbol in self.resolved_cache:
        return self.resolved_cache[symbol]
    if symbol not in self.reference_map:
        raise ResolutionError(f"{symbol} not found")
    hash_id = self.reference_map[symbol]
    if not self.cs.contains(hash_id):
        module_data = self.fetch(hash_id)
        self.cs.store(hash_id, module_data)
    module = self.cs.load(hash_id)
    self.resolved_cache[symbol] = module
    return module

resolver = Resolver(reference_map, content_store,
                    network_fetch_fn)
foo_module = resolver.resolve('foo')
```

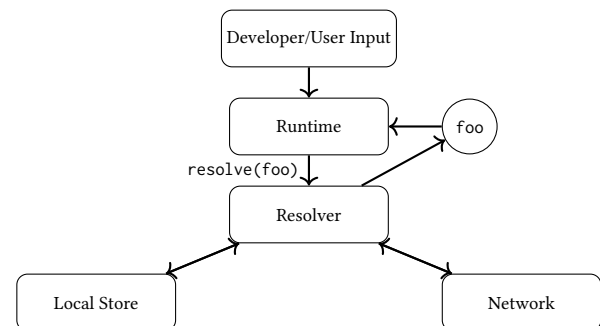


Figure 5. Runtime resolution of symbols like foo through local or network lookup.

14.4 Module Resolution Walkthrough

To illustrate the module resolution process and how conflicts are handled, consider the following scenario

involving two modules, `moduleA` and `moduleB`, both of which define a function `compute()`:

```
# moduleA.js
export function compute() { return 42; }
```

```
# moduleB.js
export function compute() { return 24; }
```

A third module, `moduleC`, imports both `moduleA` and `moduleB`, and then calls `compute()`:

```
# moduleC.js
import { compute } from 'moduleA';
import { compute } from 'moduleB';
```

```
console.log(compute());
```

Module resolution begins by evaluating imports in lexical order. In this case, the function `compute` from `moduleB` shadows the one from `moduleA` because it is imported later. Thus, `console.log(compute())` prints 24, not 42.

This rule, where later imports take precedence, is a key aspect of the system's lexical scoping model and ensures predictability: the most recently imported binding of a symbol is the one used.

Re-Exports. Now consider a re-export scenario where a module re-exports symbols from another:

```
# moduleA.js
export function compute() { return 42; }
```

```
# moduleB.js
export { compute } from 'moduleA';
export function compute() { return 24; }
```

```
# moduleC.js
import { compute } from 'moduleB';
console.log(compute());
```

Here, `moduleB` re-exports `compute` from `moduleA`, but also defines its own version of `compute`. The local definition in `moduleB` takes precedence over the re-exported one, and so `console.log(compute())` prints 24.

Nested Imports. Finally, consider a case with nested imports:

```
# base.js
export function compute() { return 1; }
```

```
# lib.js
import { compute as baseCompute } from 'base';
export function compute() {
  return baseCompute() + 1;
}
```

```
# app.js
```

```
import { compute } from 'lib';
console.log(compute());
```

In this example, `lib.js` imports `compute` from `base.js` under the alias `baseCompute`, then defines its own `compute` that builds on it. When `app.js` imports from `lib.js`, it sees only the final `compute`—which returns 2.

This demonstrates how symbol resolution in nested modules always respects lexical ordering and explicit rebinding: imported symbols are only visible where they are explicitly propagated.

Summary. These examples collectively illustrate how lexical import order and explicit rebindings guide name resolution. The "last import wins" rule ensures intuitive behavior while allowing developers to override or refine imported symbols as needed.

14.5 Execution Model Summary

Execution proceeds as follows:

1. **Resolve:** Traverse R_M to locate all ancestor modules by hash and build ρ .
2. **Instantiate:** Load and prepare Main for execution with no parent modules yet loaded.
3. **Evaluate:** As symbols are accessed, dynamically load ancestors and resolve symbols according to R_M .

This model allows large dependency graphs to be traversed efficiently and on-demand, while maintaining strong guarantees of determinism, immutability, and verifiability.

14.6 Incremental Inheritance and Differential Reuse

In immutable, content-addressed module systems, every module is identified by a cryptographic hash of its contents. This immutability provides strong guarantees—such as referential transparency, verifiable caching, and decentralized fetch—but it also presents challenges when incremental updates or overrides are needed.

Rather than allowing in-place mutation or overlays, such systems promote *differential reuse* through fine-grained modularization. Reusable logic is factored into small, composable units that can be selectively inherited and extended. When a change is necessary, a new module imports one or more ancestors and extends or overrides their contents, producing a new hash and a new node in the inheritance tree.

These systems are typically language-agnostic, deferring to the host language or runtime to resolve overlapping definitions. Resolution is often governed by a deterministic traversal of the module ancestry graph, such as a topological sort of the Merkle DAG, possibly ordered lexicographically by content hash.

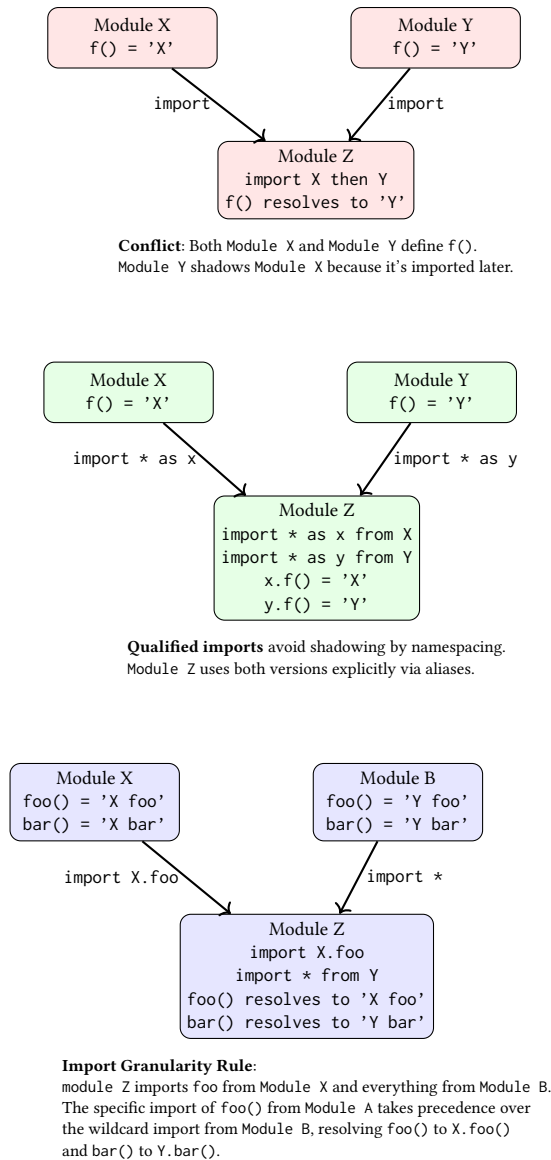


Figure 6. Module resolution: (a) Conflict resolution with lexical shadowing, where the last import wins. (b) Qualified imports preserve access to all symbols without shadowing by using aliases. (c) Demonstrating the import granularity rule: a specific import of foo from moduleA takes precedence over the general import of everything from moduleB, resolving foo() to moduleA.foo() and bar() to moduleB.bar().

This approach parallels strategies in existing systems: Git commits form immutable DAGs that track content

evolution;¹ Nix builds use content-addressed derivations and dependency graphs to isolate and reproduce environments;² and IPFS organizes files in Merkle DAGs to enable decentralized sharing and incremental updates.³ Promoting fine-grained modularization discourages monolithic library modules in favor of small, single-purpose modules that can be easily transmitted and executed on all manner of host configurations.

Future extensions may explore support for semantic diffs, patch modules, or structured deltas. These approaches could reduce duplication in the module graph, improve resolution performance, and allow for more expressive forms of inheritance without compromising immutability or verifiability.

15 Optimizations and Caching

The modular resolution model presented in this paper supports a variety of optimizations that improve both runtime performance and network efficiency, while preserving the determinism and compositionality of the system.

The primary opportunity for optimization lies in the *resolve phase*, during which symbolic references are matched against entries in the reference map and resolved to content-addressed identifiers. Because resolution is deterministic and purely functional with respect to the reference graph, the results of this phase can be cached and reused across executions, contexts, and even deployments.

To support efficient execution, the system defers the fetching and linking of modules until they are actually needed, enabling a form of *lazy loading*. Rather than eagerly loading the full transitive closure of a program's dependencies, the runtime can locate only those modules that are necessary for the execution of a given entry point. This is particularly effective in large module hierarchies or interactive environments, where only a subset of bindings are used in a given session.

The reference graph itself is well-suited to caching, both locally and in networked deployments. Because every node in the graph is identified by a content hash, any resolved module can be safely cached without risk of inconsistency. In distributed settings, resolved modules can be fetched from nearby peers or mirrors, with trust rooted in the immutability of the content hash.

¹Git's object model ensures immutability and deduplication via SHA-1/SHA-256 content hashes, enabling reproducible version histories.

²The Nix package manager treats software builds as pure functions, with inputs hashed to ensure deterministic outputs and maximal reuse.

³IPFS (InterPlanetary File System) encodes file versions and directories as Merkle trees, allowing deduplicated and verifiable content exchange.

This allows for efficient and resilient retrieval of modules without the need for centralized coordination or a trusted registry.

Symbol resolution is further optimized by exploiting structural properties of the reference map. For example, when a symbol is requested, the runtime can avoid unnecessary traversals by using a scoped cache of previously resolved bindings. Because resolution follows a lexically-scoped and shadowing-aware order, these bindings can be reused for subsequent queries in the same context.

Overall, the caching and lazy resolution strategy supports rapid execution, offline availability, and scalable distribution without compromising the reproducibility or auditability of the resulting execution. The modular and immutable nature of the reference graph allows these optimizations to be layered on top of the formal resolution semantics, rather than requiring changes to the resolution model itself.

15.1 Scalability of Cryptographic Mechanisms

While Merkle tree-based inheritance offers strong guarantees of immutability, integrity, and reproducibility, it also introduces computational overhead when applied to large module graphs. In large-scale ecosystems, the transitive ancestry of a module may span tens or hundreds of thousands of modules, each contributing to the final Merkle root.

To mitigate this cost, my system implements subtree memoization and batched ancestry hashing. Rather than recomputing the entire tree on every resolution, previously computed subtrees are cached and reused whenever their structure remains unchanged. This significantly reduces both CPU overhead and memory consumption during finalization.

Hash computation is structured as a bottom-up traversal of the module DAG, enabling parallelization across subtrees. This design integrates naturally with distributed build systems and continuous integration pipelines, where isolated subgraphs can be finalized independently and reused across builds.

Importantly, finalized modules are immutable. Their Merkle subtrees can be shared across dependents without duplication, enabling efficient reuse and dramatically reducing redundant hashing. In large registries, this reuse leads to logarithmic rather than linear growth in unique hash entries.

Future extensions may incorporate authenticated data structures, such as vector commitments or sparse Merkle trees, to compress ancestry proofs and further reduce tree depth, particularly for flat or highly reused module hierarchies.

16 Developer Tooling

A module system grounded in Merkle tree-based inheritance and content-addressed resolution introduces new interaction models for developers. This section outlines the types of tools and developer experiences needed to work effectively with such a system, especially as module graphs grow in size and complexity.

16.1 Module Graph Visualization

Inheritance and dependency graphs in Merkle-based systems can grow large and complex, particularly with deep chains of module reuse or broad forks in ancestry. Developer tooling should support interactive graph visualizations, showing both lexical import structure and resolved ancestry. This helps identify where symbols originate, which modules are shadowed, and how resolution paths are computed. Tools should allow filtering by symbol, version, or ancestry to pinpoint specific resolution behaviors.

16.2 Error Reporting and Diagnostics

When resolution fails, due to missing modules, unresolved identifiers, or hash mismatches, the system should provide rich diagnostics:

- Symbol resolution traces showing candidate paths and overrides.
- Hash mismatch explanations with expected vs. actual content.
- Resolution conflicts with suggestions for disambiguation.

Because modules are immutable, ancestry traces are often more informative than runtime stack traces. Tooling should simulate step-by-step resolution paths to clarify precedence and shadowing.

16.3 IDE and CLI Integration

Integration with modern IDEs and CLI environments is critical. Features include:

- Auto-completion for inherited symbols and re-exports.
- Hover-based ancestry and override inspection.
- Inline diffs between overridden definitions.
- Symbol provenance queries via CLI.
- Dry-run simulations of finalization and resolution behavior.

Tooling integrates with VS Code, JetBrains IDEs, and common shells, exposing both real-time editing support and CI/CD debugging aids.

16.4 Debugging Overridden Definitions

Given the potential for deep inheritance chains and override conflicts, tools should help developers reason about which definitions are active, which are overridden, and

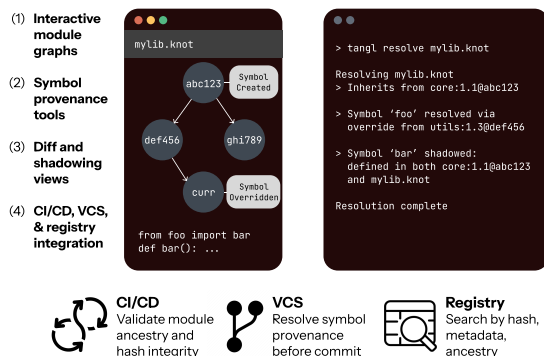


Figure 7. A conceptual overview of developer tooling for Merkle-based module systems. IDE and CLI tools provide layered insights: (1) interactive module graphs show lexical and resolved ancestry; (2) symbol provenance tools trace resolution across override chains; (3) diff and shadowing views visualize active vs overridden definitions; and (4) integration with CI/CD, VCS, and registries supports reproducibility and ecosystem interoperability.

why. Especially in the presence of complex MRO-like resolution orders, IDEs or visualization layers should explain precedence and substitution clearly. Symbol diff tools, shadowing inspectors, and override explainers can improve transparency and prevent subtle bugs due to unexpected overrides.

16.5 Visual Tooling for Symbol Overrides and Ancestry Tracing

Consider a scenario where a developer hovers over a symbol like `foo()` in an IDE. The tooling might display:

- The Merkle root or alias of the module where `foo()` was defined,
- A linear ancestry of overrides (`moduleA` → `moduleB` → `current`),
- Inline diffs for each override.

Listing 4. Mockup of a symbol override trace as presented in an IDE

```
[Trace] Symbol: foo()
Resolved to:
    moduleC@b17e... (current context)
Override history:
- moduleC@b17e...
  -> override of foo() [defined inline]
- moduleB@a9c4...
  -> re-exported from moduleA@d3e1...
- moduleA@d3e1...
  -> original definition:
```

```
function foo() { return 42; }
Ancestry path:
moduleA -> moduleB -> moduleC
```

```
- This symbol was overridden in moduleC
[INFO] Click to view diff between
moduleC::foo and moduleA::foo
```

16.6 Best Practices and Developer Guidance

To ensure predictable and maintainable behavior, developers should adhere to the following practices:

Avoid Unintentional Shadowing: Since symbol resolution follows a deterministic merge order, avoid shadowing an imported symbol unless intentionally overriding the logic for that symbol.

Use Explicit Imports for Critical Symbols: Prefer explicit imports of key symbols when re-exporting from deeply nested modules. `import(a.foo)` is better than `import(a)` if you only need to access `foo`.

Pin Modules Before Finalization: Always resolve imports to content hashes as soon as you know what functionality you need. This prevents unexpected behavior from the latest tag as new modules are uploaded.

Document Inheritance Chains: Document override purposes and expected resolution behaviors. This aids in both auditing and usage of your code.

Validate Resolution with Tooling: Use diagnostics and linters to confirm that resolution paths are deterministic.

Prefer Compositional Reuse: Shallow hierarchies simplify debugging and reduce resolution cost. Prefer direct imports over indirect ones, when possible.

17 Ecosystem Integration and Usability

Usability at scale depends not only on local developer tooling but also on system-level integration. This section discusses how Merkle-based modules interact with package managers, CI/CD workflows, and registries.

17.1 Integration with Existing Package Managers

The system is designed to interoperate with existing build and package management ecosystems. Compatibility tools can be created to allow developers to continue using `npm`, `Cargo`, or `pip` while gaining immutability and content-based resolution.

- `npm merkle-resolve` would add content hash locking to `package.json`.
- `cargo finalize` would validate dependency graphs before publishing.

- pip-merk would overlay Merkle-tracked modules into virtual environments.
- etc.

These integrations ensure a gradual adoption path for existing codebases.

17.2 Community and Ecosystem Tooling

As module systems grow beyond individual projects, ecosystem-facing tools become essential:

- Search by hash prefix, semantic version, or ancestry similarity.
- Provenance visualization and diffing between forks.
- Module discovery through public or federated registries.
- Moderated publication pipelines for namespaces and groups.

Registry APIs expose reproducibility guarantees and ancestry-based filtering, allowing deeper insights into module evolution.

17.3 UI Affordances and Developer Trust

Cryptographic guarantees are necessary but not sufficient. Tools should also surface human-readable trust signals:

- Verified authorship (e.g., PGP signatures or decentralized IDs).
- Maintainer metadata and reputation indicators.
- Visualized module forks and merge histories.
- Attestations of build reproduction or CI checks.

These affordances help developers navigate decentralized module ecosystems with greater confidence.

17.4 End-User Trust and UI Affordances

In decentralized systems where module resolution depends on transitive ancestry, trust becomes not only a cryptographic but also a human-comprehensible property. To make the Merkle-based model approachable for developers, I outline several affordances and UX principles for building trust and transparency into tooling.

Visualizing Ancestry and Inheritance. Graphical interfaces should render inheritance hierarchies as DAGs with clear origin points, version labels, and shadowing relationships. By allowing users to inspect a module's full ancestry—down to the hash of each contributing node—tools can make otherwise opaque resolution behaviors auditable and explainable. UI elements such as hover-to-expand ancestry and click-to-trace symbol origin would help users verify correctness and trace override behavior.

Conflict and Shadowing Feedback. In cases where symbols are shadowed, redefined, or inherited from conflicting sources, IDEs and CLI tools should surface this information through precise diagnostics. For example, a redefinition should include a warning with the hash and origin of the overshadowed definition, allowing users to distinguish intentional overrides from accidental conflicts.

Provenance and Content Integrity. Since all modules are content-addressed, the system guarantees immutability and integrity. To leverage this at the user level, I propose visual markers (e.g., lock icons, color-coded trust levels) indicating whether a module has a known provenance (e.g., signed by a trusted party, linked to a public repository) or is anonymously authored. This also supports audit logs and reproducibility for CI/CD workflows.

Resolution Preview and Explainability. Before committing to a resolution (e.g., during installation or CI runs), tools should offer a dry-run or preview mode that shows the exact resolved graph, ancestry paths, and expected runtime behavior. This preemptively catches surprises and builds confidence in the decentralized resolution logic.

Fallback and Error Diagnosis. If a resolution fails, due to unavailability, conflict, or missing parents, the UI should provide a structured diagnosis: which hash or module could not be fetched, what the fallback path was, and what resolution rule was violated. These diagnostics reinforce the explainability of resolution semantics and empower users to act on failure cases without guessing.

Distributed Trust as a Design Goal. Ultimately, the decentralized nature of the system allows users to define their own trust policies: whether to trust modules from known domains, require signatures from specific keys, or enforce reproducible builds. The user interface should expose these policies transparently and allow users to override, escalate, or audit as needed.

17.5 Trust, Transparency, and Developer Experience

To promote transparency and end-user trust in decentralized module systems, I propose several user interface (UI) and user experience (UX) affordances that make module ancestry, versioning, and resolution behaviors visible and auditable.

Human-Readable Provenance. Every module should expose a navigable ancestry tree derived from its Merkle DAG, displaying inherited modules, shadowed symbols, and override boundaries. This supports

debugging and helps developers audit the provenance of imported behavior.

Version Pinning Visualizations. In interfaces such as IDEs or web registries, developers should be able to inspect which module versions were finalized, whether exact content hashes or semantic ranges were used, and when hash mismatches are detected due to upstream updates.

Ancestry Conflict Warnings. When multiple inherited modules define the same symbol (e.g., via diamond inheritance or multi-parent links), IDEs and command-line tools should emit warnings or previews showing which definition will be resolved, under what rule (e.g., left-to-right linearization), and where ambiguity might arise.

Transparency in Hash-Based Locking. To reinforce the integrity guarantees of Merkle-based resolution, tooling should surface which parts of a project are hash-locked, which are open to resolution, and which inherit their lock status from parent modules. This allows users to audit potential supply chain risks at resolution time.

Diff and Override Inspection. For any module derived from one or more parents, developers should be able to diff the inherited and modified contents. This capability helps identify whether a derived module introduces functional changes, or simply re-exports behavior from its lineage.

Registry and Peer Provenance. In decentralized contexts, trust also depends on the source of modules. Tools should display metadata about which registry or peer contributed each resolved module, along with verification of its hash and ancestry. This reduces trust assumptions and helps detect impersonation or mispublishing.

These affordances are critical for developer confidence in decentralized and inheritance-based systems. While the underlying resolution machinery is content-addressed and formally sound, user interfaces must make this machinery explainable and traceable for real-world adoption.

17.6 Ecosystem Compatibility and Developer Trust

Maintaining compatibility with existing systems also helps build trust. Developers are more likely to adopt new infrastructure if it complements their existing tooling and requires minimal disruption. This includes supporting familiar metadata formats, dependency declarations, and tooling conventions. Moreover, reproducibility guarantees provided by content-addressed modules can enhance security and build integrity in existing workflows.

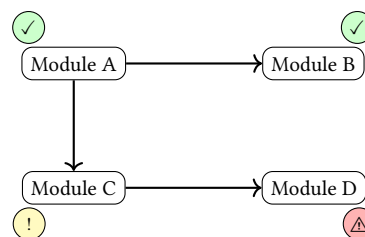


Figure 8. A trust overlay on the module graph. Each node represents a module, annotated with a trust indicator: green check for verified maintainers, yellow exclamation for unsigned modules, and red warning for known issues or missing signatures. This overlay aids users in evaluating dependency trustworthiness.

18 Integration with Existing Ecosystems

While content-addressed, Merkle-based module systems offer significant advantages in immutability, caching, and reproducibility, integration with existing language and package ecosystems remains a key challenge. Developers rarely adopt systems in isolation; instead, they require interoperability with current tools, workflows, and dependency managers.

18.1 Migration Strategies and Tooling for Legacy Codebases

To support gradual migration from semver-based systems, my approach uses bridge layers that interpret legacy version constraints (e.g., "foo": "^1.3.0") and resolve them to concrete versions using the host ecosystem's native resolver (e.g., npm, Cargo). The resulting version (e.g., 1.3.5) is then compared against the registry to map it to a finalized Merkle root.

If the registry contains a matching resolution, the import is replaced with the corresponding content hash. If no match is found, the system can fall back to the traditional package manager to fetch and hash the package. This preserves deterministic resolution while allowing continued use of familiar semver workflows.

These bridges operate strictly at the local level: legacy semver-based imports are resolved to Merkle identifiers prior to module finalization and are not stored or propagated in the DHT. This ensures that all modules published to the distributed registry use content-based addressing exclusively, maintaining consistency and determinism across the network.

The registry serves as a shared, verifiable cache of resolved legacy imports, enabling consistent and reproducible results across environments. As more packages are pinned and published in content-addressed form, projects can progressively move away from semver-based workflows without sacrificing compatibility. Because semver metadata can be optionally attached to

newly finalized modules, developers may continue to use semver constraints locally—even after dependencies have been fully resolved and published. This remains viable as long as bridge layers attempt resolution through the registry first, falling back to the host package manager only when necessary.

Migration tooling can support this process with:

- **Bridge-assisted diffing:** Detecting changes between local graphs and published Merkle mappings.
- **Semantic hashing:** Converting legacy packages into content-addressed modules with symbol-preserving boundaries.
- **Dry-run simulations:** Previewing how semver imports will resolve under Merkle semantics.

This strategy supports incremental adoption without disrupting existing toolchains, while gradually shifting dependency resolution to a globally verifiable, content-addressed model.

18.2 Incremental Migration Patterns

Migration to a content-addressed module system can proceed incrementally, without disrupting existing workflows. Adoption typically begins at leaf nodes in the dependency graph, internal libraries, stable third-party modules, or standalone utilities, which can be finalized and published independently. These modules are wrapped in Merkle-resolved snapshots and registered for reuse without requiring upstream coordination.

Consumers of finalized modules can then migrate by replacing semver imports with Merkle references. This transition requires no source changes to dependent modules and progressively improves reproducibility. Hybrid states, where some modules use semver and others use content hashes, are fully supported by the registry semantics (Section 13), enabling gradual migration across teams or repositories.

Tooling can assist by identifying unresolved dependencies, flagging mutable imports, and suggesting content-addressed upgrades. Over time, these practices yield fully Merkle-resolved graphs with deterministic builds and traceable ancestry, without invasive code rewrites.

18.3 Interoperability and Compatibility with Existing Ecosystems

My model supports seamless integration with existing package managers (e.g., npm, Cargo, pip) by translating semver constraints into content-addressed identifiers. For example, a dependency like `rand = "0.8"` is resolved against registry metadata to a specific Merkle root (e.g., `rand@0.8.5 → QmZ9...abc`), which is then used for all resolution and validation.

Integration Highlights.

- **Performance:** Semver resolution incurs overhead, but each version is uniquely content-addressed, eliminating ambiguity.
- **Compatibility:** Lockfiles are replaced by immutable bindings but can be layered atop existing tooling.
- **Usability:** Tools display semver metadata alongside hashes and offer workflows to map between them.

Design Challenges.

- **Semver Divergence:** Varying semver semantics across ecosystems complicate unified resolution.
- **Multiple Versions:** Each version resolves to a distinct Merkle root, making duplications explicit and manageable.

Bridging Strategies.

- **Pinning:** Semver ranges are resolved and pinned to Merkle hashes, enabling reproducibility and reverse lookup.
- **Annotated Metadata:** Merkle roots retain semver annotations for transparency.
- **Layered Resolution:** Local lockfiles and registry-based resolution can coexist, supporting hybrid workflows.

These mechanisms ensure a smooth migration path without compromising determinism or compatibility.

19 Interoperability and Protocols for Real-World Systems

Interoperability with existing package management systems such as npm, Cargo, and Solidity is central to the adoption of the proposed decentralized module registry. However, challenges arise when reconciling versioning schemes, particularly with respect to transitive dependencies and version mismatches between systems.

19.1 Handling Transitive Dependencies

One of the core challenges in integrating a decentralized system with traditional package managers is resolving transitive dependencies. In conventional systems like npm, dependencies often specify ranges of versions, leading to the possibility of version inflation and conflicts when dependencies overlap across packages. In contrast, my system uses deterministic Merkle hashes to resolve each dependency to a specific module version. This eliminates ambiguity but raises the need for a robust conflict resolution mechanism when transitive dependencies across systems result in mismatches.

I propose the following strategies for resolving version conflicts:

- **Priority Ordering:** When conflicts occur, the system can prioritize higher-level dependencies to avoid circular dependency resolution or duplication. Dependencies with stricter version requirements take precedence over looser ranges.
- **Range-to-Hash Mapping:** I propose a flexible mapping between semver version ranges and Merkle hashes. The system can automatically translate version ranges into the closest matching content-addressed version, allowing cross-system compatibility.
- **Conflict Resolution API:** An API will provide developers with insights into why conflicts occur and offer suggestions for resolving them. This can include proposing alternative versions, merging ranges, or creating forked dependencies.

19.2 Version Mismatch Handling

While my system ensures deterministic resolution through Merkle-based identifiers, integration with ecosystems that rely on semantic versioning introduces potential mismatches in expectations. Traditional systems like npm may resolve a version range to different concrete versions over time, whereas Merkle roots represent fixed, immutable snapshots. When bridging these systems, the challenge is not just resolution but communicating intent and compatibility clearly.

To support hybrid workflows and reduce developer confusion, I introduce the following mechanisms:

- **Version Bridge Adapters:** Adapters translate semver constraints into Merkle roots at resolution time and preserve this mapping in metadata. These bridges allow existing semver declarations to function as expected, while ensuring that once resolved, all dependencies are pinned immutably for reproducibility.
- **Compatibility Hints and Warnings:** To surface mismatches early, the system can analyze metadata and structural differences between modules resolved from semver constraints. For example, if two modules claim semver compatibility but differ significantly in their Merkle lineage, tools may flag the resolution as potentially unsafe or inconsistent with semver expectations.

These features are designed to make interoperability safer and more transparent, especially during the transition from traditional versioning to fully content-addressed module resolution.

19.3 Security Implications

With a decentralized system, the integrity and security of the module graph depend on the trustworthiness of the content hashes and the associated module metadata.

However, integrating with external systems introduces the risk of malicious updates or erroneous versioning. To address these concerns, I rely on cryptographic signatures to authenticate module sources, ensuring that only authorized publishers can publish updates. Additionally, the system can integrate with existing security protocols (e.g., npm's audit feature) to provide developers with security alerts about potential vulnerabilities in third-party modules.

20 Scalability in Large Distributed Registries

In globally distributed settings with millions of modules, scalability becomes critical. The proposed system uses a content-addressed DHT (Distributed Hash Table) for resolution, where each node can cache subtrees of the Merkle module graph. To evaluate scalability, future work should measure metrics such as:

- **Average DHT hop count** for resolution queries under varying node churn rates.
- **Cache hit rates** across popular modules, considering both first-party imports and indirect ancestry.
- **Resolution throughput** under simulated load across geographically distributed peers.

To mitigate unreliable networks or partial availability, each module's ancestry path can be requested independently in parallel, with redundancy strategies (e.g., Kademlia's k -bucket querying or erasure coding) used to increase resilience. Caching policies based on usage patterns and predictive prefetching could also reduce remote lookups.

I also plan to explore hierarchical DHT overlays and regional replication zones, allowing popular modules to propagate more aggressively in high-traffic regions. These directions will guide both practical deployment and formal scalability modeling.

Theoretically, the system's architecture benefits from logarithmic lookup time due to the DHT's structure. Because modules are content-addressed and immutable, resolution can proceed by independently fetching and caching Merkle subtrees. This allows decentralized, parallel resolution across nodes and minimizes redundant fetches even as the registry scales to millions of modules.

21 Security Considerations

The use of cryptographic hashes and Merkle trees in this system ensures that modules and their ancestry are tamper-resistant. However, there are potential attack vectors to consider:

Hash Collision Attacks: While cryptographic hash functions such as SHA-256 are widely considered secure, hash collisions could theoretically lead to multiple different modules having the same identifier. To mitigate this risk, I recommend using SHA-256 or higher, which offers a large enough output space to make collisions highly improbable. Additionally, I use Merkle trees to provide a unique fingerprint of module ancestry, making it extremely difficult to alter a module's content without detection. Finally, both the registry and the module store enforce immutability: once a module or registry entry is created, it cannot be overridden or edited. As a result, any attempt to insert a hash-colliding module would fail, preserving integrity and preventing ambiguous resolutions.

Sybil Attacks in the Registry: To ensure the integrity of the distributed registry, I employ public key cryptography to sign namespace Merkle roots, ensuring that only the owner of a namespace can modify its entries. Clients must verify these signatures before accepting updates, protecting against unauthorized modifications.

Replay Attacks: To prevent replay attacks, where an attacker might reuse a previously finalized module in an unintended context, each finalization event includes a cryptographically signed timestamp and context-specific metadata. This binding ensures that a module finalized for one purpose or environment cannot be silently replayed in another without detection. Any mismatch in context or timestamp would invalidate the signature, making unauthorized reuse evident and verifiable.

DoS Resilience: Content-addressed module systems can mitigate denial-of-service (DoS) attacks, such as hash bomb or modules with excessive fan-out, by enforcing configurable limits on resolution depth and import breadth. These thresholds can be set based on empirical norms observed across healthy module graphs, such as median fan-out per module or typical dependency tree depth in a given ecosystem. Modules exceeding these bounds may be rejected or require explicit overrides. Lazy resolution strategies further enhance resilience by fetching only the required subtrees of the dependency graph, limiting exposure to malformed or excessive structures.

21.1 Expanded Security Threat Model

In addition to the attack methods already discussed, I also consider network partitioning and module unavailability as potential security risks. To mitigate partitioning, the system utilizes a decentralized module resolution approach, which allows nodes to resolve modules

independently, even when parts of the network are unavailable.

Moreover, the system's caching mechanism is designed to prevent exploitation. If a malicious actor attempts to cache incorrect or outdated modules, the system checks the integrity of cached modules using cryptographic proofs before allowing their use.

I also address potential exploits related to module unavailability. If a module is unavailable due to network failure or malicious interference, the system gracefully falls back to local caches, and the developer is notified with an appropriate error message, ensuring minimal disruption.

21.2 Future-Proofing Hash Function Security

While SHA-256 currently provides strong preimage and collision resistance, long-term security requires anticipating future cryptographic threats, including those posed by quantum computing. Post-quantum cryptographic research has produced several candidate hash functions, such as those based on the SPHINCS+ and XMSS families, which offer resistance to quantum attacks at the cost of larger output sizes and slower performance.

Future work includes evaluating these alternative hashing algorithms for suitability in content-addressed module systems, focusing on compatibility with Merkle tree constructions, registry performance, and adoption viability. Additionally, migration strategies may be developed to support gradual transition between hash functions, including hybrid hash encoding schemes or dual-hash identifiers that preserve backward compatibility while enabling forward security. By future-proofing the core cryptographic primitives of the system, I aim to maintain integrity guarantees even in the face of emerging threats.

22 CI/CD and VCS Integration (Supplementary)

A detailed discussion of how the proposed system integrates with continuous integration (CI) workflows and version control systems (VCS), including semantic conflict detection, immutable tagging, and cache-aware builds, is included in the supplementary materials (Appendix G).

23 Evaluation and Testing Methodology

I outline the evaluation goals and empirical validation plan for my Merkle-based module system. While a full implementation is still in development, I report preliminary findings based on synthetic data and provide a roadmap for testing real-world performance, scalability, and usability. Complete benchmark configurations,

Table 1. Synthetic Benchmark Summary

Benchmark	Setup	Result
Finalization Time	Linear chains (depth N)	Linear scaling in N
Symbol Lookup Latency	Repeated lookups in cached graphs	Sublinear (amortized) due to caching
Graph Compression	Diamond graphs with shared ancestors	60–80% subtree reuse

generated results, and synthetic graph structures are provided in the supplementary materials.

23.1 Evaluation Goals

My evaluation strategy targets the following questions:

- How does resolution performance scale with graph width, depth, and ancestry reuse?
- How robust is the system under partial failures or adversarial input?
- Can inheritance semantics preserve determinism and modularity in practice?

These goals inform the benchmarks and testing workflows I prioritize.

23.2 Prototype Development

I will implement a reference prototype in Rust [17] using libp2p [9] for decentralized communication and Wasmer [18] for WebAssembly execution. This prototype will serve as the basis for full-scale empirical validation.

23.3 Future Testing Plan

Empirical testing will evaluate:

- Latency and memory usage across large dependency graphs.
- Resolution correctness under stale caches and node dropout.
- Conflict handling in symbol shadowing and multi-parent inheritance.
- Developer-facing tooling for visualization, debugging, and traceability.

Testing will simulate real-world workflows (e.g., CI/CD, interactive development) to assess usability and performance.

23.4 Supplementary Benchmarks and Testing Scenarios

Detailed benchmark results from synthetic module graphs, as well as comparative analyses with existing tools (e.g., npm, Cargo, pip), are available in the supplementary materials. These include:

- Synthetic benchmarks on finalization, symbol resolution, and structural compression.
- Stress tests for network partitioning, stale ancestry, and fallback performance.

- Metrics such as average DHT hops, cache hit rates, and deduplication efficiency.
- Simulated workflows including dependency graph rebuilding and distributed editing.

These results offer preliminary insights into the system's expected behavior and scalability.

Supplementary Materials. Due to space constraints, full details of my synthetic benchmark methodology, prototype development plan, and empirical evaluation scenarios are provided in the supplementary materials. These include synthetic results, proposed metrics, testing workflows, and comparative analysis with existing systems.

24 Limitations and Future Work

Although the Merkle tree-based inheritance model provides strong guarantees of determinism, integrity, and compositionality, several limitations remain:

- **Performance on modules with many direct parents.** Finalization hashes the set of direct parent IDs. A large number of parents—now including atomic groups—can increase sorting and hashing overhead.
- **Recursive groups complexity.** SCC detection and group serialization introduce additional build-time work. Extremely large recursive groups may challenge memory and canonical ordering.
- **Registry dependency.** Symbolic imports rely on a distributed registry. Network failures or malicious updates can block resolution unless clients maintain robust caches and fallback strategies.
- **Limited language-specific typing.** We rely on side-car metadata for exported symbol types. Full support for parametric polymorphism, type classes, and richer type-level constraints remains to be integrated.
- **Tooling maturity.** IDE plugins, visual diff tools, and override inspectors exist only as prototypes. Production-ready developer experiences are essential for adoption.

24.1 Future Work

To address these limitations and broaden the model's applicability, I plan to explore:

- **Authenticated data-structure optimizations.** Leverage sparse Merkle trees or vector commitments to compress large parent sets and support constant-size proofs.
- **Graph deltas and patch modules.** Introduce differential inheritance modules that encode only changes relative to a parent, reducing duplication and speeding up finalization.
- **Richer type systems.** Formalize and mechanize generic type aliases, subtype constraints, and module-level interface contracts within the finalization framework.
- **Zero-knowledge proofs of lineage.** Enable privacy-preserving verification by proving ancestry inclusion without revealing module contents.
- **Robust registry protocols.** Design decentralized consensus or reputation schemes to guard against Sybil attacks, equivocation, and unauthorized updates.
- **Compilers and linkers as modules.** Model compiler frontends, optimizers, and linkers as Merkle-rooted modules, enabling fully decentralized, reproducible build pipelines in which each compilation stage is content-addressed and verifiable.
- **Ecosystem integration.** Develop production-grade adapters for npm, Cargo, pip, CI/CD systems, and IDEs to support incremental migration and seamless workflows.

25 Conclusion

I have presented a Merkle tree-based inheritance model that treats each module as a cryptographic commitment to both its code and its resolved dependencies. By hashing only the finalized parent IDs, each of which already embeds its full ancestry, finalization remains efficient, deterministic, and tamper-proof. Symbol resolution follows a fixed, breadth-first merge order, producing unambiguous override traces and enabling fine-grained module composition without mutable registries or lock-files.

My prototype implementation, Tangl, demonstrates how these ideas can integrate with existing ecosystems via semver translation adapters and decentralized registries. I have also stated the core metatheoretic properties of determinism, injectivity, and compositionality in the main text.

For complete pseudocode of the finalization algorithm, mechanized proofs of the lean theorems, and

a formal mapping to existing module calculi, please consult the supplementary materials.

References

- [1] Davide Ancona and Elena Zucca. 2002. Foundations of Component-Based Systems. *ACM Computing Surveys* 34, 1 (2002), 1–31.
- [2] Juan Benet. 2014. IPFS - Content Addressed, Versioned, P2P File System. arXiv:1407.3561 [cs.NI] <https://arxiv.org/abs/1407.3561> Accessed: 2025-04-09.
- [3] Giovanni Castagna. 1993. The Mixin Calculus: Its Theory and Applications. *Journal of Functional Programming* 3, 4 (1993), 517–568. doi:10.1017/S095679680000146X
- [4] Scott Chacon and Ben Straub. 2014. *Pro Git*. Apress, Berkeley, CA.
- [5] Rust Project Developers. 2025. The Cargo Book. <https://doc.rust-lang.org/cargo/>. Accessed: 2025-04-09.
- [6] Eelco Dolstra. 2006. Nix (package manager). <https://nixos.org/manual/nix/>.
- [7] Derek Dreyer, Karl Cray, and Robert Harper. 2003. A Type System for Higher-Order Modules. In *Proceedings of the 30th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL '03)*. ACM, New York, NY, 236–249. doi:10.1145/604131.604151
- [8] S. C. Dunn. 2025. Tangl: A Distributed Module Network with Dual Execution Environments. (2025). Tangl white paper, available upon request.
- [9] libp2p. 2025. libp2p: A Modular Network Stack. <https://libp2p.io/> Accessed: 2025-04-13.
- [10] Ralph C. Merkle. 1978. Secure Communications Over Insecure Channels. *Proceedings of the IEEE International Conference on Communications* 13 (1978), 33–36.
- [11] Ralph C. Merkle. 1987. A Digital Signature Based on a Conventional Encryption Function. In *A Conference on the Theory and Applications of Cryptographic Techniques on Advances in Cryptology (CRYPTO '87)*. Springer-Verlag, Berlin, Heidelberg, 369–378.
- [12] John C. Mitchell. 1988. Modules and Abstract Types. In *Proceedings of the ACM SIGPLAN '88 Conference on Programming Language Design and Implementation (PLDI)*. ACM, New York, NY, 252–261. doi:10.1145/539133.539159
- [13] Inc. npm. 2025. npm Documentation. <https://docs.npmjs.com/about-npm>. Accessed: 2025-04-09.
- [14] Alan Snyder. 1986. Encapsulation and Inheritance in Object-Oriented Programming Languages. In *Conference Proceedings on Object-Oriented Programming Systems, Languages and Applications (OOPSLA '86)*. ACM, New York, NY, USA, 38–45. doi:10.1145/28697.28702
- [15] Ion Stoica, Robert Morris, David Liben-Nowell, David R. Karger, M. Frans Kaashoek, Frank Dabek, and Hari Balakrishnan. 2003. Chord: A Scalable Peer-to-Peer Lookup Protocol for Internet Applications. *IEEE/ACM Trans. Netw.* 11, 1 (Feb. 2003), 17–32. doi:10.1109/TNET.2002.808407
- [16] The pip Developers. 2025. pip documentation. <https://pip.pypa.io/en/stable/>.
- [17] The Rust Programming Language. 2025. Rust Programming Language. <https://www.rust-lang.org/> Accessed: 2025-04-13.
- [18] Wasmer. 2025. Wasmer: The Universal WebAssembly Runtime. <https://wasmer.io/> Accessed: 2025-04-13.